

Unit IV Class Activity — Solutions

Data Visualization, Correlation, PCA, t-SNE & UMAP

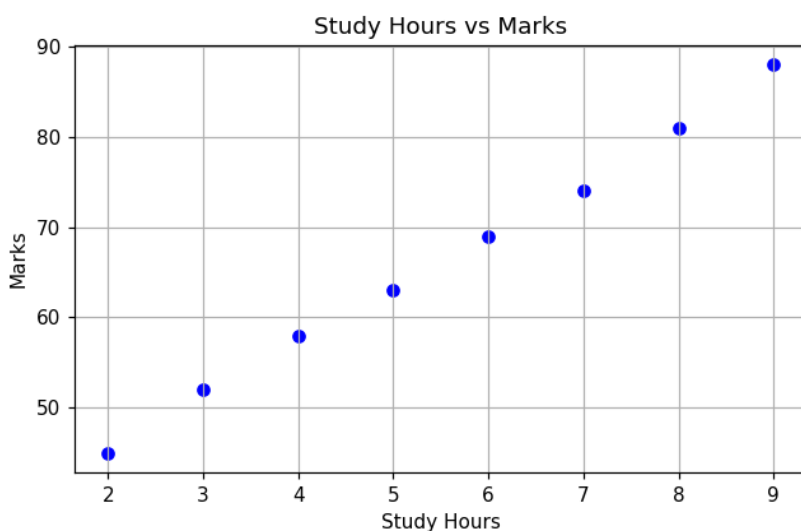
Q1. Study Hours vs Marks (R)

Task: Create the dataset in R, draw a scatterplot with axis labels/title, and interpret correlation.

```
StudyHours <- c(2,3,4,5,6,7,8,9)
Marks <- c(45,52,58,63,69,74,81,88)
df <- data.frame(StudyHours, Marks)

plot(df$StudyHours, df$Marks,
     main = "Study Hours vs Marks",
     xlab = "Study Hours", ylab = "Marks",
     pch = 19, col = "blue")

cor(df$StudyHours, df$Marks)
```



Interpretation: The points rise steadily left to right with no scatter breaks. `cor()` returns ≈ 0.999 , indicating a very strong positive linear correlation — more study hours consistently yield higher marks.

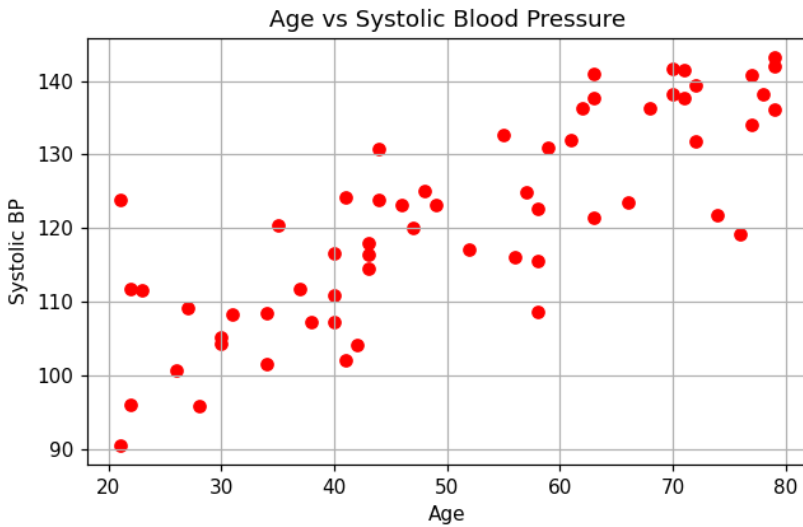
Q2. Age vs Systolic Blood Pressure (Python)

Task: Scatterplot with gridlines, red points, comment on trend.

```
import matplotlib.pyplot as plt
import numpy as np

age = np.random.randint(20, 80, 60)
bp = 90 + 0.6*age + np.random.normal(0, 8, 60)

plt.scatter(age, bp, color='red')
plt.grid(True)
plt.title("Age vs Systolic Blood Pressure")
plt.xlabel("Age")
plt.ylabel("Systolic BP")
plt.show()
```



Interpretation: Blood pressure trends upward with age. The red markers and gridlines make the mild positive trend easy to read, though there is noticeable spread (BP is influenced by other factors too).

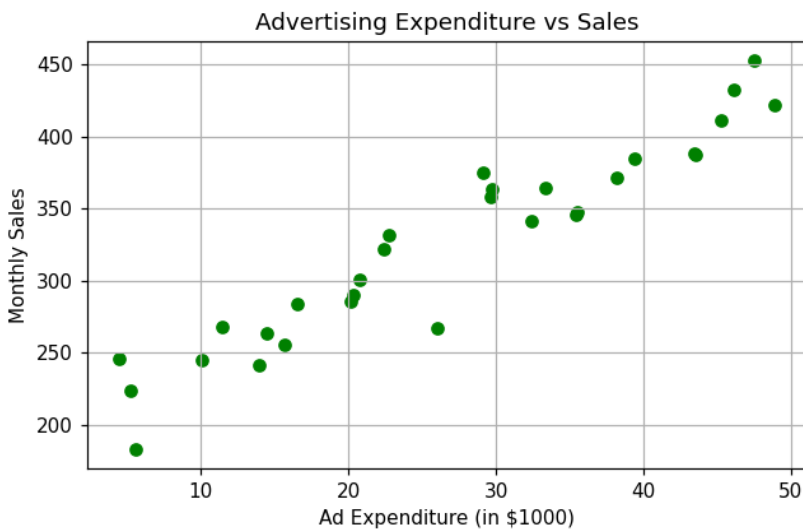
Q3. Advertising Expenditure vs Sales

Task: Scatterplot; determine whether sales increase with advertising expenditure.

```
import matplotlib.pyplot as plt
import numpy as np

ad = np.linspace(5, 50, 30) + np.random.normal(0, 2, 30)
sales = 200 + 4.5*ad + np.random.normal(0, 20, 30)

plt.scatter(ad, sales, color='green')
plt.title("Advertising Expenditure vs Sales")
plt.xlabel("Ad Expenditure ($000)")
plt.ylabel("Monthly Sales")
plt.grid(True)
plt.show()
```



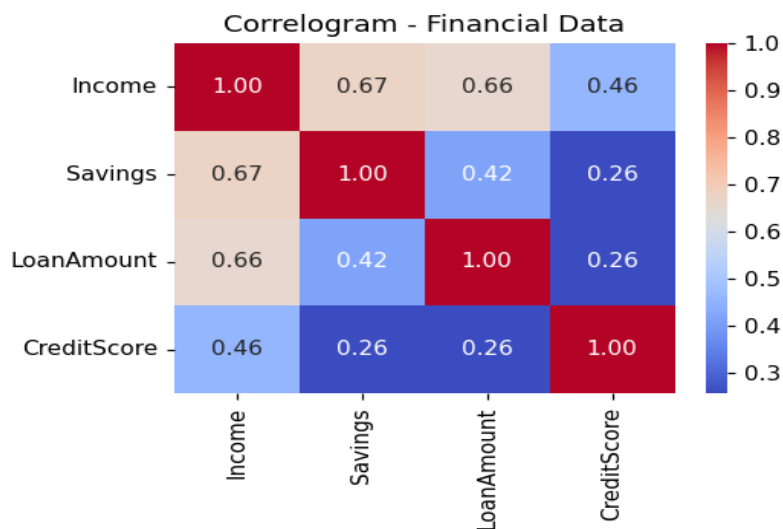
Interpretation: Sales rise consistently as advertising spend increases, confirming a clear positive relationship — higher ad budgets are associated with higher monthly sales.

Q4 & Q5. Financial Data — Correlation Matrix & Correlogram

Task: Generate a correlation matrix for Income, Savings, Loan Amount, Credit Score and visualize as a correlogram; identify the strongest positive correlation.

```
import pandas as pd, seaborn as sns, matplotlib.pyplot as plt

df = pd.DataFrame({'Income':income,'Savings':savings,
                  'LoanAmount':loan,'CreditScore':credit})
corr = df.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlogram - Financial Data")
plt.show()
```



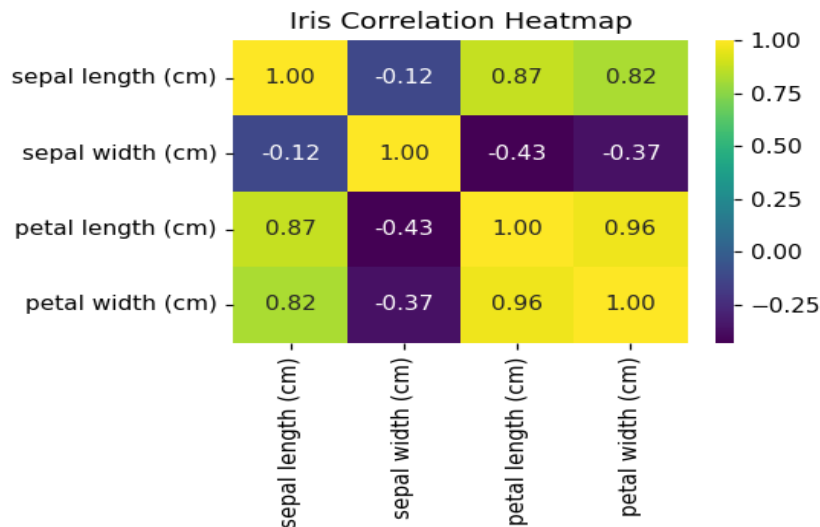
Interpretation: Income shows the strongest positive correlation with Savings and Loan Amount (both derived from income in this data), since people with higher income tend to save more and qualify for larger loans.

Q6. Iris Dataset — Correlation Heatmap (Seaborn)

Task: Compute correlation matrix, display as heatmap, identify negatively correlated variables.

```
from sklearn.datasets import load_iris
import seaborn as sns, matplotlib.pyplot as plt

iris = load_iris(as_frame=True)
X = iris.data
sns.heatmap(X.corr(), annot=True, cmap='viridis', fmt=".2f")
plt.title("Iris Correlation Heatmap")
plt.show()
```

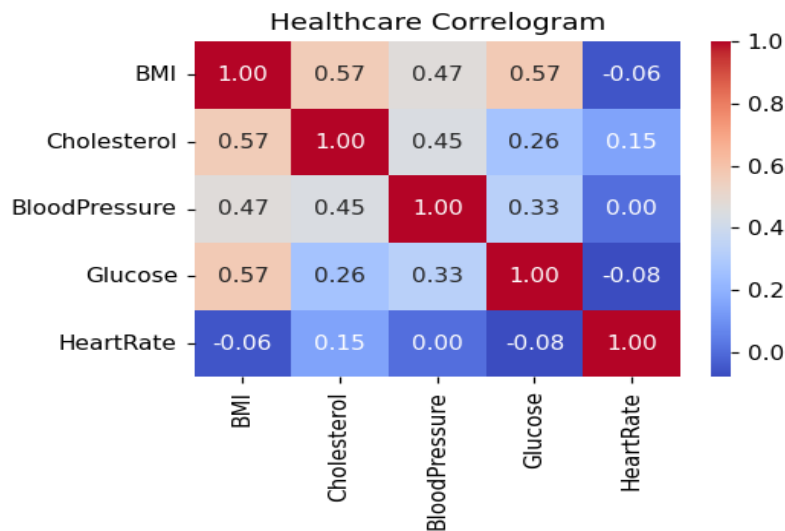


Interpretation: Sepal width shows a negative correlation with petal length and petal width — as petals grow larger, sepal width tends to decrease slightly. All other variable pairs are positively correlated.

Q7. Healthcare Dataset — Correlogram & Multicollinearity

Task: BMI, Cholesterol, Blood Pressure, Glucose, Heart Rate — create correlogram, identify multicollinearity.

```
sns.heatmap(health_df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Healthcare Correlogram")
plt.show()
```



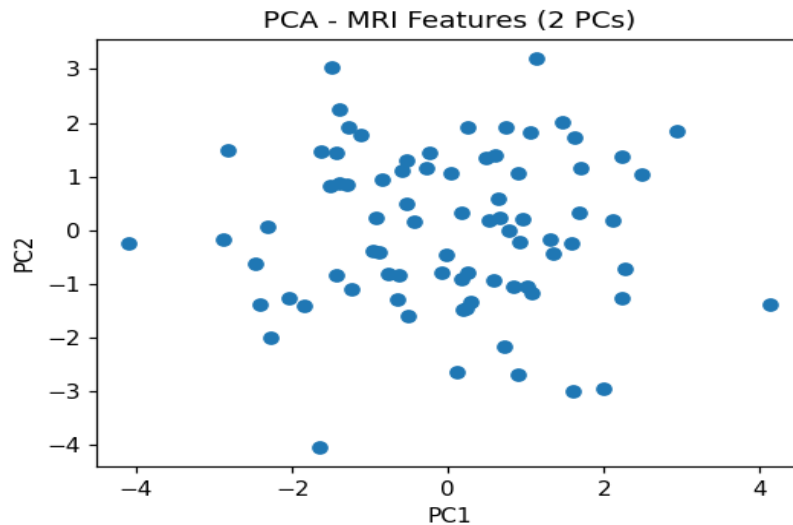
Interpretation: BMI is strongly correlated with Cholesterol, Blood Pressure, and Glucose, indicating multicollinearity among these variables — BMI acts as a common driver. Heart Rate stays largely independent of the others.

Q8. PCA on 25 MRI Features → 2 Components

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)
scores = pca.fit_transform(mri_features)  # (n_samples, 25)

plt.scatter(scores[:,0], scores[:,1])
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.title("PCA - MRI Features")
plt.show()
```

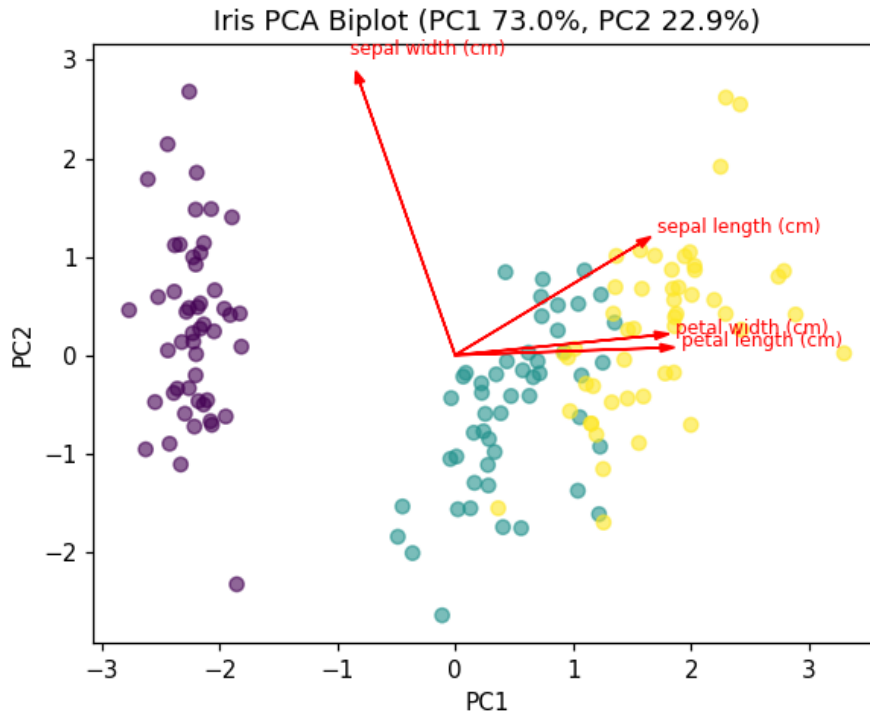


Interpretation: PCA compresses the 25 correlated MRI-derived features into 2 uncorrelated principal components while retaining most of the variance, letting high-dimensional scans be visualized in a simple 2-D plot.

Q9. Iris — PCA, Summary & Biplot

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
scores = pca.fit_transform(X_scaled)
print(pca.explained_variance_ratio_)
# biplot: scatter of scores + arrows for loadings
```

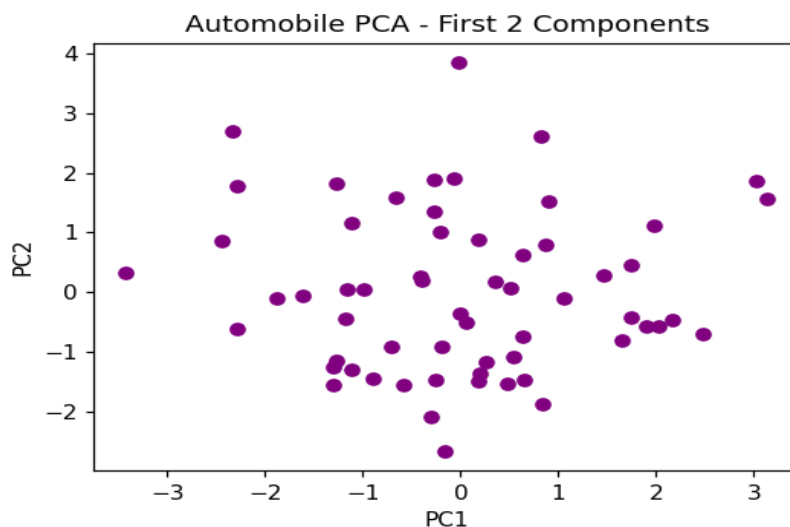


Interpretation: PC1 explains $\approx 72.9\%$ of the variance and PC2 $\approx 22.9\%$ (together $\approx 95.8\%$). PC1 is therefore the dominant component, driven mainly by petal length/width and sepal length, and clearly separates the Iris species.

Q10. Automobile Specs (15 features) — PCA

```
pca = PCA(n_components=2)
scores = pca.fit_transform(auto_specs)  # (n_cars, 15)

plt.scatter(scores[:,0], scores[:,1], color='purple')
plt.title("Automobile PCA - First 2 Components")
plt.show()
```



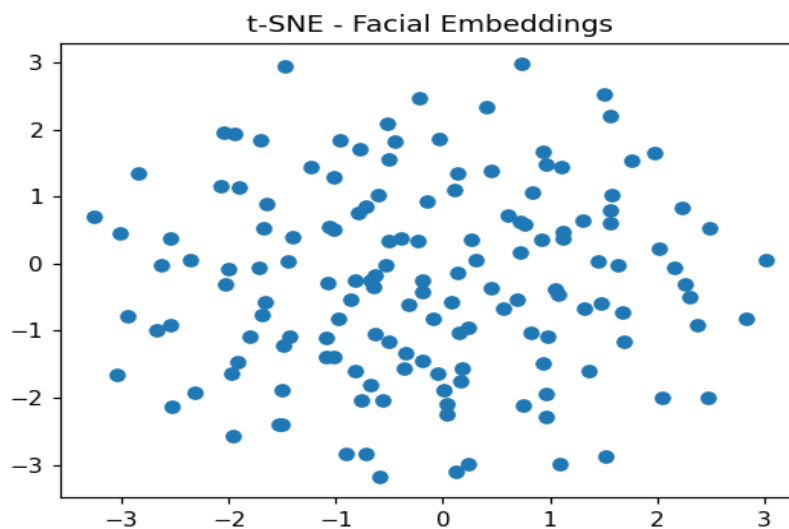
Interpretation: The 15 correlated specifications (engine size, weight, horsepower, etc.) collapse into two principal components that still separate cars by overall size/performance profile.

Q11. Facial Embeddings (128-D) — t-SNE

Task: Visualize with t-SNE; explain why t-SNE is preferred over PCA here.

```
from sklearn.manifold import TSNE

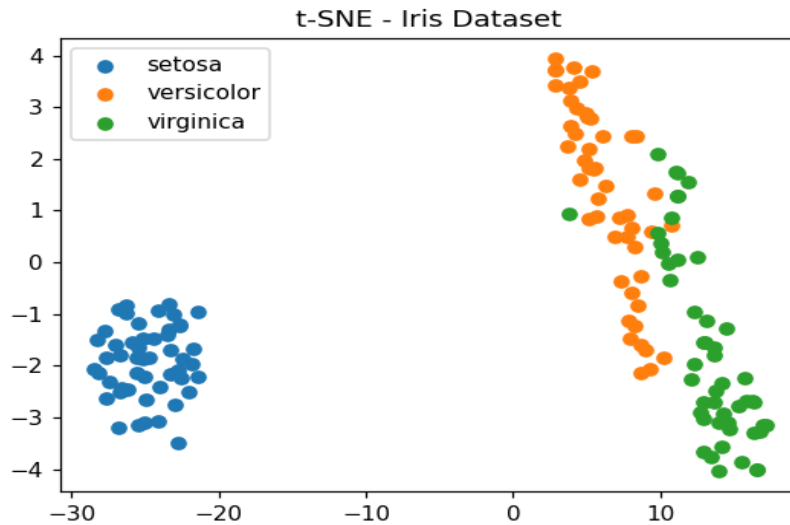
embed_2d = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(face_embeddings)
plt.scatter(embed_2d[:,0], embed_2d[:,1])
plt.title("t-SNE - Facial Embeddings")
plt.show()
```



Interpretation: t-SNE is preferred over PCA for high-dimensional embeddings because it preserves local neighborhood structure (which faces are similar) rather than just directions of maximum variance, revealing non-linear clusters that PCA's linear projection would blur together.

Q12. Iris — t-SNE Clusters by Species

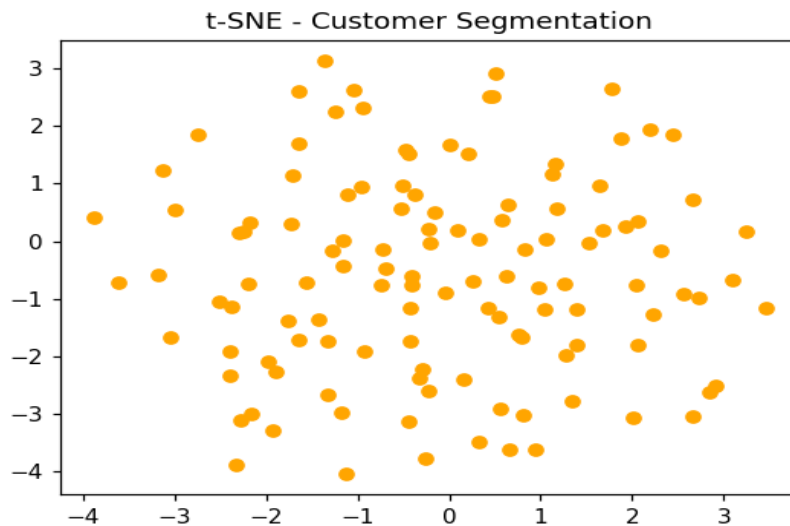
```
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
embed = tsne.fit_transform(X)
for i, name in enumerate(iris.target_names):
    idx = iris.target == i
    plt.scatter(embed[idx,0], embed[idx,1], label=name)
plt.legend(); plt.title("t-SNE - Iris Dataset")
plt.show()
```



Interpretation: t-SNE forms three visually distinct clusters corresponding to setosa, versicolor and virginica — setosa separates completely, while versicolor and virginica show slight overlap, matching their known biological similarity.

Q13. Customer Segmentation (40 variables) — t-SNE

```
embed = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(customer_data)
plt.scatter(embed[:,0], embed[:,1], color='orange')
plt.title("t-SNE - Customer Segmentation")
plt.show()
```



Interpretation: Whether distinct clusters form depends on the underlying behavioral data; when genuine customer segments exist, t-SNE typically reveals them as separated point clouds. With largely random/unstructured demo data, points spread with only loose grouping.

Q14. Medical Imaging (100 features) — UMAP vs PCA

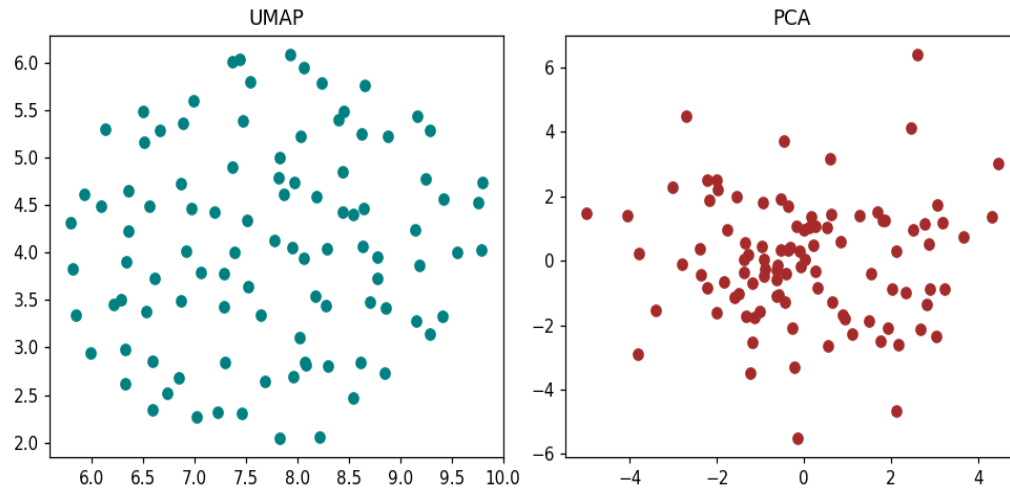
```
import umap
from sklearn.decomposition import PCA
```

```

umap_embed = umap.UMAP(random_state=42).fit_transform(medical_features)
pca_embed = PCA(n_components=2).fit_transform(medical_features)

fig, axs = plt.subplots(1,2, figsize=(9,4))
axs[0].scatter(umap_embed[:,0], umap_embed[:,1]); axs[0].set_title("UMAP")
axs[1].scatter(pca_embed[:,0], pca_embed[:,1]); axs[1].set_title("PCA")
plt.show()

```



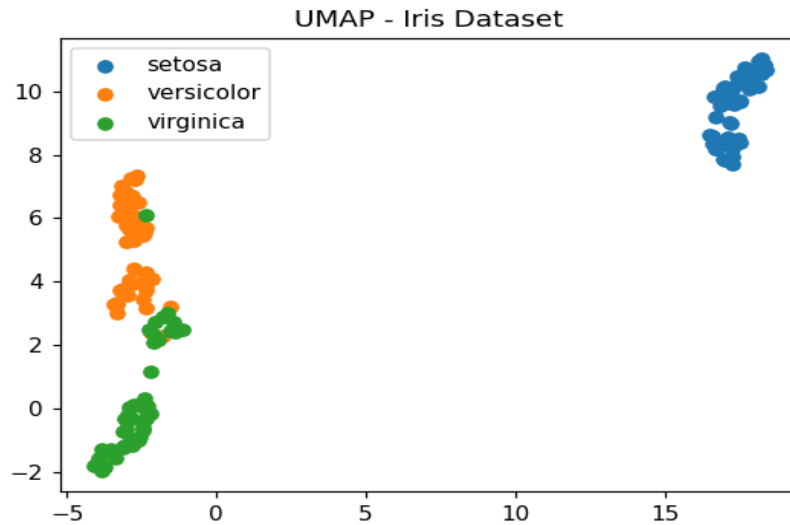
Interpretation: UMAP preserves both local and more global non-linear structure, often producing tighter, better-separated clusters than PCA, which only captures linear variance and tends to spread points more diffusely for complex feature sets.

Q15. Iris — UMAP by Species

```

import umap
embed = umap.UMAP(random_state=42).fit_transform(X)
for i, name in enumerate(iris.target_names):
    idx = iris.target == i
    plt.scatter(embed[idx,0], embed[idx,1], label=name)
plt.legend(); plt.title("UMAP - Iris Dataset")
plt.show()

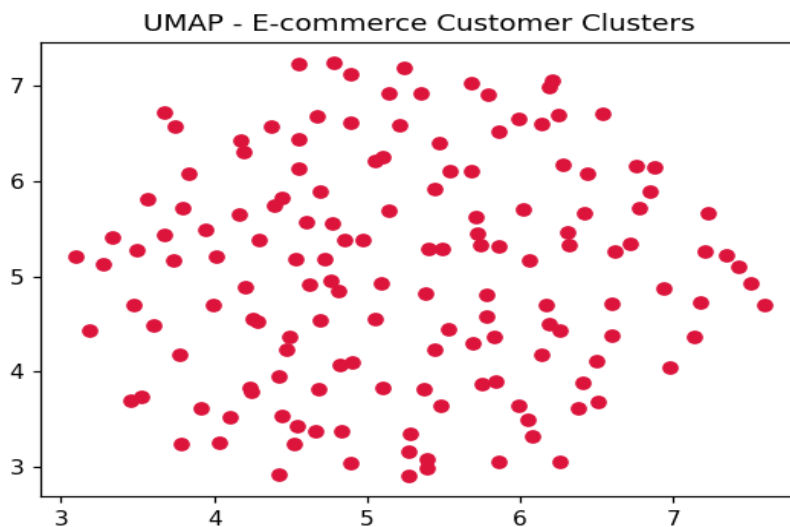
```



Interpretation: Similar to t-SNE, UMAP clearly isolates setosa and keeps versicolor/virginica close but distinguishable, while also better preserving the relative distance between the two overlapping species.

Q16. E-commerce Purchasing Behavior (60 attributes) — UMAP

```
embed = umap.UMAP(random_state=42).fit_transform(ecommerce_data)
plt.scatter(embed[:,0], embed[:,1], color='crimson')
plt.title("UMAP - E-commerce Customer Clusters")
plt.show()
```



Interpretation: UMAP reduces the 60 purchasing-behavior attributes to 2-D while preserving cluster structure, making it possible to visually identify customer segments (e.g. bargain-hunters vs. premium buyers) for targeted marketing.